

Object storage and caching for HTTP services using Redis and S3

Activity data

Activity number	4
Professor	Ramon Amela Milian
Teaching language	English
Association	Individual
Delivery date	13-07-2026

Activity description

In this activity, alternative storage systems will be worked on. The student will learn how to use caching systems with Redis and object storage with S3 (AWS simple storage service). The starting point will be a minimal running FastAPI project, and some exercises will be done to achieve a simple but complete, clean, and scalable API. More precisely, we will develop the persistence for the tokens and files. Finally, some testing suites will be used to ensure that the project works as expected.

It is recommended to start developing from the result obtained in the third deliverable so that the result is better. Nevertheless, it is possible to start with the initial template, and all the missing pieces from the first, second and third deliverable will not be considered nor penalized. More precisely, what will be evaluated in this deliverable is the implementation of completely interchangeable services through dependency injection. More precisely, Redis as cache and S3 as object storage. This kind of storage system is typical of cloud systems due to their high scalability.

Competences

CE3 – Design and manage information systems for data treatment and storage

CE6 – Process big amounts of data to extract information that helps making decisions

COMPETENCES (CE / CT)	LEARNING RESULTS (RA)	EVALUATION (IE)	INDICATORS
CE3	RA1 – Summary the historical evolution and the current opportunities in the data system administration in cloud environments	IE2 – The student can publish the project in an efficient and maintainable way	
CE6	RA6 – Assess the suitability, use and characteristics of different cloud services and solutions to solve a specific data processing problem	IE8 – The student can design an API that interacts with an object storage system	
		IE9 – The student can design an API that interacts with a cache system	
	RA8 – Plan the student's own learning needs and choose an appropriate specialization or master's degree program with a high level of autonomy	IE10 – The student can design a system with several storage systems, understanding their differences and best use cases	

Activity contents

- Redis persistence service
- S3 persistence service
- Docker image broadcast
- Github actions

Task descriptions

- Create two apps called “authentication” and “files”
- Create routers inside both new applications
- Import the new routers from the main file
- Create four folders inside each one of the apps:
 - api
 - dependency_injection
 - domain
 - persistence
- Create the following endpoints in the newly created folders.

In the following definitions, it is possible to identify the user with a string or with an email. Choose what you think is the best option.

It is important to note that always that the session token is passed as input, it should be sent through a header called “Auth”.

- **Authentication**
 - register – POST

Input: at least, the minimum data that you consider necessary to code all the other endpoints.

Action: create a new user unto the database

Output: code errors for the considered cases (for example, 200 for success, 409 when user already exists).

- login – POST

Input: user id and password.

Action: create a new session for this user

Output: connection token and code for success/errors

- logout – POST

Input: connection token

Action: close the session associated to the input token

Output: code for success/errors

- introspect – GET

Input: session token

Action: no action

Output: verify whether the token is correct or not and, in case it is, return the information associated to the user as well as the code for success/errors

- **Files**

- files – GET

Input: session token

Action: no effect

Output: return the information for all the files owned by the user corresponding to the token passed in the call and the code for success/errors

- files – POST

Input: file information and session token

Action: create a new file

Output: return the id for the created file and the code for success/errors

- files/{id} – GET

Input: file id and session token

Action: no action

Output: return the information (and content if present) for the id given in the input and the code for success/errors

- files/{id} – DELETE

Input: file id and session token

Action: delete the file with the id passed as parameter

Output: code for success/errors

- files/{id} – POST

Input: file id, session token and file content

Action: fill in the file content

Output: code for success/errors

- files/merge – POST

Input: several file ids, session token

Action: merge pdf's

Output: the id of the merged file and code for success/errors

It is important to note that there are two different endpoints to create files. One to upload the file information (like filename, description... and whatever information wants to be saved) and one to upload the file content.

The API should contain the routers. The domain should have the code containing the endpoint logics (token verifications, consistency checks...) and the business objects. Persistence should contain the class implementing all persistence operations to interact with the Redis and S3 storage systems. Finally, singletons linking the API with the domain in will be placed in dependency_injection.

More precisely, the following steps are necessary to develop both storage systems.

Regarding Redis, it will be used to store the session tokens. This will enable adding more than one worker and take advantage of the expiration time feature.

- Redis:
 - o Add a new target into the docker-compose
 - o Create a new persistence file in the folder to store the Redis code that will interact with the database
 - o Create dependency injection files and inject them in the files that will use this persistence (the ones in the authentication part).

S3 will be used to store the content of the files. This is a common use case. Indeed, S3 allows to create sharable URL to ease the download of the files without overcharging the server.

- S3
 - o Add a new target into the docker-compose
 - o Create a new persistence file in the folder to store the S3 code that will interact with the storage system
 - o Create dependency injection files and inject them in the endpoints handling the file content (both loading, download and deletion).

Once this has been done, the last step would be to deploy our full application. This can be done by publishing the docker image and through GitHub actions. Follow the next steps:

- Publish the project image in docker hub
- Add a GitHub action to check the format each time there is a pull request

All the previous steps could be separated in the following summary points:

1. Use Redis as caching system
2. Use S3 as object storage system
3. Publish containers so they are publicly accessible
4. Use GitHub actions to perform CI/CD automatic actions

Formal aspects of tasks

All the project's code must be delivered in compressed format through the virtual campus. In addition, in the delivered text, the commit hash with the code in *GitHub* should be included. This hash should contain the same exact version as the delivered code.

Task	Percentage
1	35%
2	40%
3	10%
4	15%



Evaluation

Number	Task	Weight	IE
4	Object storage and caching for HTTP services using Redis and S3	25%	IE2, IE8, IE9, IE10

Information resources

Capsules and videoconferences